

■ AutoParts ERP

Complete Source Code & Project Guide

.NET 10 Web API • React 19 + Vite • SQL Server

Mohammed Jamir Uddin
Full-Stack Developer | Bangladesh

Phase 1 Starter	49 Files	.NET 10	React 19	SQL Server
-----------------	----------	---------	----------	------------

Table of Contents

- 1. Project Overview
- 2. Folder Structure
- 3. Development Roadmap
- 4. Source Files
- 5. README.md
- 6. backend/AutoPartsERP.API/AutoPartsERP.API.csproj
- 7. backend/AutoPartsERP.API/appsettings.json
- 8. backend/AutoPartsERP.API/Program.cs
- 9. backend/AutoPartsERP.API/Data/AppDbContext.cs
- 10. backend/AutoPartsERP.API/Models/User.cs
- 11. backend/AutoPartsERP.API/Models/Role.cs
- 12. backend/AutoPartsERP.API/Models/Customer.cs
- 13. backend/AutoPartsERP.API/Models/Supplier.cs
- 14. backend/AutoPartsERP.API/Models/Category.cs
- 15. backend/AutoPartsERP.API/Models/Brand.cs
- 16. backend/AutoPartsERP.API/Models/Unit.cs
- 17. backend/AutoPartsERP.API/Models/Vehicle.cs
- 18. backend/AutoPartsERP.API/Models/Warehouse.cs
- 19. backend/AutoPartsERP.API/Models/Part.cs
- 20. backend/AutoPartsERP.API/Models/StockMovement.cs
- 21. backend/AutoPartsERP.API/Models/Purchase.cs
- 22. backend/AutoPartsERP.API/Models/PurchaseItem.cs
- 23. backend/AutoPartsERP.API/Models/Sale.cs
- 24. backend/AutoPartsERP.API/Models/SaleItem.cs
- 25. backend/AutoPartsERP.API/Models/Expense.cs
- 26. backend/AutoPartsERP.API/Models/Income.cs
- 27. backend/AutoPartsERP.API/Models/JournalEntry.cs
- 28. backend/AutoPartsERP.API/Models/ActivityLog.cs
- 29. backend/AutoPartsERP.API/Controllers/Auth/AuthController.cs
- 30. backend/AutoPartsERP.API/Controllers/Dashboard/DashboardController.cs
- 31. backend/AutoPartsERP.API/Controllers/Customers/CustomersController.cs
- 32. backend/AutoPartsERP.API/Controllers/Parts/PartsController.cs
- 33. database/AutoPartsERP.sql
- 34. frontend/package.json
- 35. frontend/vite.config.js
- 36. frontend/index.html
- 37. frontend/src/main.jsx
- 38. frontend/src/index.css

- 39. frontend/src/App.jsx
- 40. frontend/src/api/api.js
- 41. frontend/src/context/AuthContext.jsx
- 42. frontend/src/layouts/MainLayout/MainLayout.jsx
- 43. frontend/src/layouts/MainLayout/MainLayout.css
- 44. frontend/src/layouts/Sidebar/Sidebar.jsx
- 45. frontend/src/layouts/Sidebar/Sidebar.css
- 46. frontend/src/layouts/Navbar/Navbar.jsx
- 47. frontend/src/layouts/Navbar/Navbar.css
- 48. frontend/src/pages/Login/Login.jsx
- 49. frontend/src/pages/Login/Login.css
- 50. frontend/src/pages/Dashboard/Dashboard.jsx
- 51. frontend/src/pages/Dashboard/Dashboard.css
- 52. frontend/src/pages/Customer/CustomerList.jsx
- 53. frontend/src/pages/Parts/PartList.jsx

1. Project Overview

AutoParts ERP is a commercial-grade Enterprise Resource Planning system designed for auto parts shops, dealers, garages, and multi-branch businesses. Built with modern technologies, it covers inventory, sales, purchase, double-entry accounting, HR/payroll, and management reporting.

Tech Stack

Layer	Technology
Backend	.NET 10 Web API (C#)
ORM	Entity Framework Core
Database	SQL Server
Authentication	JWT (user-secrets)
Frontend	React 19 + Vite
Routing	React Router v7
HTTP Client	Axios
Charts	Recharts
PDF Export	QuestPDF (Phase 4)
Excel Export	ClosedXML (Phase 4)
Testing	xUnit + Moq (Phase 4)

Key Modules

Dashboard — Live KPIs: today sales, purchases, low-stock alerts

Authentication — JWT login, role-based access (Admin/Manager/Salesman/Cashier)

Customers & Suppliers — Full CRUD, opening balance tracking

Parts / Products — Barcode support, category, brand, unit, low-stock flag

Inventory — Stock In/Out/Transfer/Damage, multi-warehouse

Purchase — Purchase orders, GRN, supplier payments, returns

Sales / POS — Invoice, discount, customer due, receipt print

Accounting — Income, Expense, Journal, Ledger, Trial Balance, P&L, Balance Sheet

HR — Employees, Attendance, Payroll, Leave

Reports — Daily/Monthly/Yearly, PDF & Excel export

Settings & Backup — System config, database backup & restore

Activity Log — Full audit trail of all user actions

2. Folder Structure

```
AutoPartsERP/
├── backend/
│   ├── AutoPartsERP.API/
│   ├── Controllers/
│   │   ├── Auth/AuthController.cs
│   │   ├── Dashboard/DashboardController.cs
│   │   ├── Customers/CustomersController.cs
│   │   ├── Parts/PartsController.cs
│   │   ├── (+ 27 more controller folders)
│   ├── Models/
│   │   ├── User.cs, Role.cs, Customer.cs, Supplier.cs
│   │   ├── Part.cs, Category.cs, Brand.cs, Unit.cs
│   │   ├── Purchase.cs, PurchaseItem.cs
│   │   ├── Sale.cs, SaleItem.cs
│   │   ├── Expense.cs, Income.cs, JournalEntry.cs
│   │   ├── StockMovement.cs, Warehouse.cs, Vehicle.cs
│   │   ├── ActivityLog.cs
│   ├── Data/AppDbContext.cs
│   ├── DTOs/ (Customer/, Supplier/, Part/, ...)
│   ├── Services/ (interfaces + implementations)
│   ├── Repository/
│   ├── Middleware/ Helpers/ Mapping/ Validation/
│   ├── Program.cs
│   ├── appsettings.json
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   ├── package.json
│   ├── src/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── index.css
│   │   ├── api/api.js
│   │   ├── context/AuthContext.jsx
│   │   ├── layouts/
│   │   │   ├── MainLayout/
│   │   │   ├── Sidebar/
│   │   │   ├── Navbar/
│   │   ├── pages/
│   │   │   ├── Login/ Dashboard/ Customer/ Parts/
│   │   │   ├── (+ 35 more page folders)
│   │   ├── components/
│   │   │   ├── Table/ Form/ Card/ Chart/ Modal/
│   │   │   ├── Scanner/ InvoicePrint/ Loader/ Toast/
├── database/
│   ├── AutoPartsERP.sql (all tables + seed data)
└── README.md
```

3. Development Roadmap

Phase 1 — Foundation (STARTER INCLUDED)

- ✓ JWT Auth + Login page
- ✓ Dashboard (live stats)
- ✓ Customers CRUD
- ✓ Suppliers CRUD
- ✓ Parts + Barcode search
- ✓ Categories, Brands, Units
- ✓ Warehouses
- ✓ AppDbContext + all Models
- ✓ SQL seed data

Phase 2 — Inventory

- ✓ Stock In / Out / Transfer / Damage
- ✓ Barcode & QR scanner (USB + camera)
- ✓ Multi-warehouse stock view
- ✓ Low-stock alerts
- ✓ Stock adjustment

Phase 3 — Purchase & Sales

- ✓ Purchase orders + GRN
- ✓ Purchase returns
- ✓ Supplier payments
- ✓ POS / Sales invoice
- ✓ Discount & VAT
- ✓ Customer due management
- ✓ Sales returns
- ✓ Receipt print

Phase 4 — Accounting

- ✓ Income & Expense
- ✓ Cash Book & Bank
- ✓ Double-entry Journal
- ✓ Ledger with running balance
- ✓ Trial Balance
- ✓ Profit & Loss
- ✓ Balance Sheet
- ✓ Chart of Accounts

Phase 5 — HR

- ✓ Employee records

- ✓ Attendance
- ✓ Payroll generation
- ✓ Leave management

Phase 6 — Reports

- ✓ Daily / Monthly / Yearly reports
- ✓ PDF export (QuestPDF)
- ✓ Excel export (ClosedXML)
- ✓ Dashboard charts (Recharts)

Phase 7 — Advanced

- ✓ Multi-branch / Multi-warehouse
- ✓ Database backup & restore
- ✓ Email / SMS notifications
- ✓ Cloud deployment
- ✓ Mobile app (Flutter)

4. Source Files

Every file below is the complete, ready-to-use source code. Copy each file to the corresponding path in your project.

■ README.md

```
# AutoParts ERP

A commercial-grade Auto Parts ERP System built with .NET 10 Web API + React + Vite + SQL Server.

## Modules
Dashboard, Customers, Suppliers, Parts, Inventory, Purchase, Sales, Accounting (Income/Expense/Journal/Ledger), Reports, HR, Settings, Activity Log

## Quick Start

### 1. Database
Open SQL Server Management Studio and run: `database/AutoPartsERP.sql`

### 2. Backend
```
cd backend/AutoPartsERP.API
dotnet user-secrets init
dotnet user-secrets set "Jwt:Key" "your-64-char-random-secret"
dotnet ef migrations add Init
dotnet ef database update
dotnet run --urls "http://localhost:5151"
```

### 3. Frontend
```
cd frontend
npm install
npm run dev
```

### 4. Login
- URL: http://localhost:5173
- Email: admin@autoparts.local
- Password: Admin123!

## Tech Stack
- Backend: .NET 10 Web API, EF Core, SQL Server, JWT
- Frontend: React 19, Vite, Axios, React Router v7, Recharts

## Development Roadmap
- Phase 1: Auth, Dashboard, Master data (Customers, Suppliers, Parts) ■ Started
- Phase 2: Inventory, Barcode Scanner
- Phase 3: Purchase & Sales (POS)
- Phase 4: Accounting (Ledger, Journal, P&L, Balance Sheet)
- Phase 5: HR (Employee, Payroll, Attendance)
- Phase 6: Reports (PDF, Excel)
- Phase 7: Multi-branch, Backup, Cloud
```

■ backend/AutoPartsERP.API/AutoPartsERP.API.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
```

```

<UserSecretsId>autoparts-erp-secrets</UserSecretsId>
</PropertyGroup>
<ItemGroup>
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.0" />
<PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.0" />
<PackageReference Include="Swashbuckle.AspNetCore" Version="7.3.1" />
<PackageReference Include="BCrypt.Net-Next" Version="4.0.3" />
<PackageReference Include="ClosedXML" Version="0.104.2" />
</ItemGroup>
</Project>

```

■ backend/AutoPartsERP.API/appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost\\SQLEXPRESS;Database=AutoPartsERP;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Issuer": "AutoPartsERP",
    "Audience": "AutoPartsERPUsers",
    "ExpiryMinutes": 60
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

■ backend/AutoPartsERP.API/Program.cs

```

using AutoPartsERP.API.Data;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Database
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// JWT Authentication
var jwtKey = builder.Configuration["Jwt:Key"]
?? throw new InvalidOperationException("JWT Key is not configured. Use: dotnet user-secrets set \"Jwt:Key\" \"<your-secret>\"");

```

```

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
        };
    });

builder.Services.AddAuthorization();

// CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    policy.WithOrigins("http://localhost:5173")
    .AllowAnyHeader()
    .AllowAnyMethod());
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowFrontend");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

■ backend/AutoPartsERP.API/Data/AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;

namespace AutoPartsERP.API.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    // Security
    public DbSet<Models.User> Users => Set<Models.User>();
    public DbSet<Models.Role> Roles => Set<Models.Role>();
    public DbSet<Models.ActivityLog> ActivityLogs => Set<Models.ActivityLog>();

    // Master

```

```

public DbSet<Models.Customer> Customers => Set<Models.Customer>();
public DbSet<Models.Supplier> Suppliers => Set<Models.Supplier>();
public DbSet<Models.Category> Categories => Set<Models.Category>();
public DbSet<Models.Brand> Brands => Set<Models.Brand>();
public DbSet<Models.Unit> Units => Set<Models.Unit>();
public DbSet<Models.Vehicle> Vehicles => Set<Models.Vehicle>();
public DbSet<Models.Part> Parts => Set<Models.Part>();
public DbSet<Models.Warehouse> Warehouses => Set<Models.Warehouse>();

// Inventory
public DbSet<Models.StockMovement> StockMovements => Set<Models.StockMovement>();

// Purchase
public DbSet<Models.Purchase> Purchases => Set<Models.Purchase>();
public DbSet<Models.PurchaseItem> PurchaseItems => Set<Models.PurchaseItem>();

// Sales
public DbSet<Models.Sale> Sales => Set<Models.Sale>();
public DbSet<Models.SaleItem> SaleItems => Set<Models.SaleItem>();

// Accounting
public DbSet<Models.Expense> Expenses => Set<Models.Expense>();
public DbSet<Models.Income> Incomes => Set<Models.Income>();
public DbSet<Models.JournalEntry> JournalEntries => Set<Models.JournalEntry>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
}
}

```

■ backend/AutoPartsERP.API/Models/User.cs

```

namespace AutoPartsERP.API.Models;
public class User
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public int RoleId { get; set; }
    public Role? Role { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/Role.cs

```

namespace AutoPartsERP.API.Models;
public class Role
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
}

```

■ backend/AutoPartsERP.API/Models/Customer.cs

```
namespace AutoPartsERP.API.Models;
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Phone { get; set; }
    public string? Email { get; set; }
    public string? Address { get; set; }
    public decimal OpeningBalance { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

■ backend/AutoPartsERP.API/Models/Supplier.cs

```
namespace AutoPartsERP.API.Models;
public class Supplier
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Phone { get; set; }
    public string? Email { get; set; }
    public string? Address { get; set; }
    public decimal OpeningBalance { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

■ backend/AutoPartsERP.API/Models/Category.cs

```
namespace AutoPartsERP.API.Models;
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
}
```

■ backend/AutoPartsERP.API/Models/Brand.cs

```
namespace AutoPartsERP.API.Models;
public class Brand
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Country { get; set; }
}
```

■ backend/AutoPartsERP.API/Models/Unit.cs

```
namespace AutoPartsERP.API.Models;
public class Unit
```

```
{
public int Id { get; set; }
public string Name { get; set; } = string.Empty;
public string? Symbol { get; set; }
}
```

■ backend/AutoPartsERP.API/Models/Vehicle.cs

```
namespace AutoPartsERP.API.Models;
public class Vehicle
{
public int Id { get; set; }
public string Make { get; set; } = string.Empty;
public string Model { get; set; } = string.Empty;
public int? Year { get; set; }
public string? Type { get; set; }
}
```

■ backend/AutoPartsERP.API/Models/Warehouse.cs

```
namespace AutoPartsERP.API.Models;
public class Warehouse
{
public int Id { get; set; }
public string Name { get; set; } = string.Empty;
public string? Location { get; set; }
public bool IsActive { get; set; } = true;
}
```

■ backend/AutoPartsERP.API/Models/Part.cs

```
namespace AutoPartsERP.API.Models;
public class Part
{
public int Id { get; set; }
public string Name { get; set; } = string.Empty;
public string? PartNumber { get; set; }
public string? Barcode { get; set; }
public int? CategoryId { get; set; }
public Category? Category { get; set; }
public int? BrandId { get; set; }
public Brand? Brand { get; set; }
public int? UnitId { get; set; }
public Unit? Unit { get; set; }
public decimal CostPrice { get; set; }
public decimal SalePrice { get; set; }
public int ReorderLevel { get; set; }
public int CurrentStock { get; set; }
public bool IsActive { get; set; } = true;
}
```

■ backend/AutoPartsERP.API/Models/StockMovement.cs

```

namespace AutoPartsERP.API.Models;
public class StockMovement
{
    public int Id { get; set; }
    public int PartId { get; set; }
    public Part? Part { get; set; }
    public int WarehouseId { get; set; }
    public Warehouse? Warehouse { get; set; }
    public string Type { get; set; } = string.Empty; // IN, OUT, TRANSFER, DAMAGE
    public int Quantity { get; set; }
    public string? Reference { get; set; }
    public string? Notes { get; set; }
    public DateTime MovedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/Purchase.cs

```

namespace AutoPartsERP.API.Models;
public class Purchase
{
    public int Id { get; set; }
    public string InvoiceNo { get; set; } = string.Empty;
    public int SupplierId { get; set; }
    public Supplier? Supplier { get; set; }
    public DateTime PurchaseDate { get; set; }
    public decimal TotalAmount { get; set; }
    public decimal PaidAmount { get; set; }
    public decimal DueAmount => TotalAmount - PaidAmount;
    public string Status { get; set; } = "Pending";
    public List<PurchaseItem> Items { get; set; } = new();
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/PurchaseItem.cs

```

namespace AutoPartsERP.API.Models;
public class PurchaseItem
{
    public int Id { get; set; }
    public int PurchaseId { get; set; }
    public Purchase? Purchase { get; set; }
    public int PartId { get; set; }
    public Part? Part { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
    public decimal TotalPrice => Quantity * UnitPrice;
}

```

■ backend/AutoPartsERP.API/Models/Sale.cs

```

namespace AutoPartsERP.API.Models;
public class Sale
{
    public int Id { get; set; }
    public string InvoiceNo { get; set; } = string.Empty;
    public int? CustomerId { get; set; }
}

```

```

public Customer? Customer { get; set; }
public DateTime SaleDate { get; set; }
public decimal TotalAmount { get; set; }
public decimal Discount { get; set; }
public decimal PaidAmount { get; set; }
public decimal DueAmount => TotalAmount - Discount - PaidAmount;
public string Status { get; set; } = "Completed";
public List<SaleItem> Items { get; set; } = new();
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/SaleItem.cs

```

namespace AutoPartsERP.API.Models;
public class SaleItem
{
    public int Id { get; set; }
    public int SaleId { get; set; }
    public Sale? Sale { get; set; }
    public int PartId { get; set; }
    public Part? Part { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
    public decimal TotalPrice => Quantity * UnitPrice;
}

```

■ backend/AutoPartsERP.API/Models/Expense.cs

```

namespace AutoPartsERP.API.Models;
public class Expense
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Category { get; set; }
    public decimal Amount { get; set; }
    public DateTime ExpenseDate { get; set; }
    public string? Notes { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/Income.cs

```

namespace AutoPartsERP.API.Models;
public class Income
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Source { get; set; }
    public decimal Amount { get; set; }
    public DateTime IncomeDate { get; set; }
    public string? Notes { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

■ backend/AutoPartsERP.API/Models/JournalEntry.cs

```
namespace AutoPartsERP.API.Models;
public class JournalEntry
{
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal DebitAmount { get; set; }
    public decimal CreditAmount { get; set; }
    public string AccountHead { get; set; } = string.Empty;
    public DateTime EntryDate { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

■ backend/AutoPartsERP.API/Models/ActivityLog.cs

```
namespace AutoPartsERP.API.Models;
public class ActivityLog
{
    public int Id { get; set; }
    public int? UserId { get; set; }
    public string Action { get; set; } = string.Empty;
    public string? EntityName { get; set; }
    public string? Details { get; set; }
    public DateTime LoggedAt { get; set; } = DateTime.UtcNow;
}
```

■ backend/AutoPartsERP.API/Controllers/Auth/AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using AutoPartsERP.API.Data;

namespace AutoPartsERP.API.Controllers.Auth;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _db;
    private readonly IConfiguration _config;

    public AuthController(AppDbContext db, IConfiguration config)
    {
        _db = db;
        _config = config;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest req)
    {

```

```

var user = await _db.Users
    .Include(u => u.Role)
    .FirstOrDefaultAsync(u => u.Email == req.Email && u.IsActive);

if (user == null || !BCrypt.Net.BCrypt.Verify(req.Password, user.PasswordHash))
    return Unauthorized(new { message = "Invalid credentials" });

var token = GenerateToken(user);
return Ok(new { token, user.FullName, role = user.Role?.Name });
}

private string GenerateToken(models.User user)
{
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Role, user.Role?.Name ?? "")
    };

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(
            double.Parse(_config["Jwt:ExpiryMinutes"] ?? "60")),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}

public record LoginRequest(string Email, string Password);

```

■ backend/AutoPartsERP.API/Controllers/Dashboard/DashboardController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AutoPartsERP.API.Data;

namespace AutoPartsERP.API.Controllers.Dashboard;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class DashboardController : ControllerBase
{
    private readonly AppDbContext _db;
    public DashboardController(AppDbContext db) => _db = db;

    [HttpGet]
    public async Task<IActionResult> GetSummary()

```

```

{
var today = DateTime.Today;
var monthStart = new DateTime(today.Year, today.Month, 1);

var todaySales = await _db.Sales
.Where(s => s.SaleDate.Date == today)
.SumAsync(s => (decimal?)s.TotalAmount) ?? 0;

var todayPurchases = await _db.Purchases
.Where(p => p.PurchaseDate.Date == today)
.SumAsync(p => (decimal?)p.TotalAmount) ?? 0;

var monthSales = await _db.Sales
.Where(s => s.SaleDate >= monthStart)
.SumAsync(s => (decimal?)s.TotalAmount) ?? 0;

var lowStockCount = await _db.Parts
.CountAsync(p => p.IsActive && p.CurrentStock <= p.ReorderLevel);

var totalCustomers = await _db.Customers.CountAsync(c => c.IsActive);
var totalSuppliers = await _db.Suppliers.CountAsync(s => s.IsActive);
var totalParts = await _db.Parts.CountAsync(p => p.IsActive);

return Ok(new
{
today = new { sales = todaySales, purchases = todayPurchases },
thisMonth = new { sales = monthSales },
alerts = new { lowStock = lowStockCount },
totals = new { customers = totalCustomers, suppliers = totalSuppliers, parts = totalParts }
});
}
}

```

■ backend/AutoPartsERP.API/Controllers/Customers/CustomersController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AutoPartsERP.API.Data;
using AutoPartsERP.API.Models;

namespace AutoPartsERP.API.Controllers.Customers;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class CustomersController : ControllerBase
{
private readonly AppDbContext _db;
public CustomersController(AppDbContext db) => _db = db;

[HttpGet]
public async Task<IActionResult> GetAll() =>
Ok(await _db.Customers.Where(c => c.IsActive).ToListAsync());

[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)

```

```

{
var c = await _db.Customers.FindAsync(id);
return c == null ? NotFound() : Ok(c);
}

[HttpPost]
public async Task<IActionResult> Create(Customer customer)
{
_db.Customers.Add(customer);
await _db.SaveChangesAsync();
return CreatedAtAction(nameof(GetById), new { id = customer.Id }, customer);
}

[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, Customer customer)
{
if (id != customer.Id) return BadRequest();
_db.Entry(customer).State = EntityState.Modified;
await _db.SaveChangesAsync();
return NoContent();
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
var c = await _db.Customers.FindAsync(id);
if (c == null) return NotFound();
c.IsActive = false;
await _db.SaveChangesAsync();
return NoContent();
}
}

```

■ backend/AutoPartsERP.API/Controllers/Parts/PartsController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AutoPartsERP.API.Data;
using AutoPartsERP.API.Models;

namespace AutoPartsERP.API.Controllers.Parts;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class PartsController : ControllerBase
{
private readonly AppDbContext _db;
public PartsController(AppDbContext db) => _db = db;

[HttpGet]
public async Task<IActionResult> GetAll() =>
Ok(await _db.Parts
.Include(p => p.Category)
.Include(p => p.Brand)
.Include(p => p.Unit)

```

```

.Where(p => p.IsActive)
.Select(p => new {
p.Id, p.Name, p.PartNumber, p.Barcode,
Category = p.Category!.Name,
Brand = p.Brand!.Name,
Unit = p.Unit!.Name,
p.CostPrice, p.SalePrice,
p.CurrentStock, p.ReorderLevel,
LowStock = p.CurrentStock <= p.ReorderLevel
})
.ToListAsync());

[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
var p = await _db.Parts.FindAsync(id);
return p == null ? NotFound() : Ok(p);
}

[HttpGet("barcode/{barcode}")]
public async Task<IActionResult> GetByBarcode(string barcode)
{
var p = await _db.Parts.FirstOrDefaultAsync(x => x.Barcode == barcode);
return p == null ? NotFound() : Ok(p);
}

[HttpPost]
public async Task<IActionResult> Create(Part part)
{
_db.Parts.Add(part);
await _db.SaveChangesAsync();
return CreatedAtAction(nameof(GetById), new { id = part.Id }, part);
}

[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, Part part)
{
if (id != part.Id) return BadRequest();
_db.Entry(part).State = EntityState.Modified;
await _db.SaveChangesAsync();
return NoContent();
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
var p = await _db.Parts.FindAsync(id);
if (p == null) return NotFound();
p.IsActive = false;
await _db.SaveChangesAsync();
return NoContent();
}
}

```

```

-- AutoPartsERP Database
-- Run this in SQL Server Management Studio

CREATE DATABASE AutoPartsERP;
GO
USE AutoPartsERP;
GO

-- Security
CREATE TABLE Roles (Id INT PRIMARY KEY IDENTITY, Name NVARCHAR(50) NOT NULL);
CREATE TABLE Users (
  Id INT PRIMARY KEY IDENTITY,
  FullName NVARCHAR(100) NOT NULL,
  Email NVARCHAR(100) NOT NULL UNIQUE,
  PasswordHash NVARCHAR(255) NOT NULL,
  RoleId INT REFERENCES Roles(Id),
  IsActive BIT DEFAULT 1,
  CreatedAt DATETIME DEFAULT GETDATE()
);

-- Master
CREATE TABLE Customers (
  Id INT PRIMARY KEY IDENTITY,
  Name NVARCHAR(100) NOT NULL,
  Phone NVARCHAR(20),
  Email NVARCHAR(100),
  Address NVARCHAR(255),
  OpeningBalance DECIMAL(18,2) DEFAULT 0,
  IsActive BIT DEFAULT 1,
  CreatedAt DATETIME DEFAULT GETDATE()
);
CREATE TABLE Suppliers (
  Id INT PRIMARY KEY IDENTITY,
  Name NVARCHAR(100) NOT NULL,
  Phone NVARCHAR(20),
  Email NVARCHAR(100),
  Address NVARCHAR(255),
  OpeningBalance DECIMAL(18,2) DEFAULT 0,
  IsActive BIT DEFAULT 1,
  CreatedAt DATETIME DEFAULT GETDATE()
);
CREATE TABLE Categories (Id INT PRIMARY KEY IDENTITY, Name NVARCHAR(100) NOT NULL, Description NVARCHAR(255));
CREATE TABLE Brands (Id INT PRIMARY KEY IDENTITY, Name NVARCHAR(100) NOT NULL, Country NVARCHAR(50));
CREATE TABLE Units (Id INT PRIMARY KEY IDENTITY, Name NVARCHAR(50) NOT NULL, Symbol NVARCHAR(10));
CREATE TABLE Vehicles (Id INT PRIMARY KEY IDENTITY, Make NVARCHAR(50), Model NVARCHAR(50), Year INT, Type NVARCHAR(50));
CREATE TABLE Warehouses (Id INT PRIMARY KEY IDENTITY, Name NVARCHAR(100) NOT NULL, Location NVARCHAR(255), IsActive BIT DEFAULT 1);

CREATE TABLE Parts (
  Id INT PRIMARY KEY IDENTITY,
  Name NVARCHAR(150) NOT NULL,
  PartNumber NVARCHAR(50),
  Barcode NVARCHAR(50),
  CategoryId INT REFERENCES Categories(Id),
  BrandId INT REFERENCES Brands(Id),

```

```

UnitId INT REFERENCES Units(Id),
CostPrice DECIMAL(18,2) DEFAULT 0,
SalePrice DECIMAL(18,2) DEFAULT 0,
ReorderLevel INT DEFAULT 5,
CurrentStock INT DEFAULT 0,
IsActive BIT DEFAULT 1
);

-- Inventory
CREATE TABLE StockMovements (
Id INT PRIMARY KEY IDENTITY,
PartId INT REFERENCES Parts(Id),
WarehouseId INT REFERENCES Warehouses(Id),
Type NVARCHAR(20) NOT NULL, -- IN, OUT, TRANSFER, DAMAGE, RETURN
Quantity INT NOT NULL,
Reference NVARCHAR(50),
Notes NVARCHAR(255),
MovedAt DATETIME DEFAULT GETDATE()
);

-- Purchase
CREATE TABLE Purchases (
Id INT PRIMARY KEY IDENTITY,
InvoiceNo NVARCHAR(30) NOT NULL,
SupplierId INT REFERENCES Suppliers(Id),
PurchaseDate DATE NOT NULL,
TotalAmount DECIMAL(18,2) DEFAULT 0,
PaidAmount DECIMAL(18,2) DEFAULT 0,
Status NVARCHAR(20) DEFAULT 'Pending',
CreatedAt DATETIME DEFAULT GETDATE()
);
CREATE TABLE PurchaseItems (
Id INT PRIMARY KEY IDENTITY,
PurchaseId INT REFERENCES Purchases(Id),
PartId INT REFERENCES Parts(Id),
Quantity INT NOT NULL,
UnitPrice DECIMAL(18,2) NOT NULL
);

-- Sales
CREATE TABLE Sales (
Id INT PRIMARY KEY IDENTITY,
InvoiceNo NVARCHAR(30) NOT NULL,
CustomerId INT REFERENCES Customers(Id),
SaleDate DATE NOT NULL,
TotalAmount DECIMAL(18,2) DEFAULT 0,
Discount DECIMAL(18,2) DEFAULT 0,
PaidAmount DECIMAL(18,2) DEFAULT 0,
Status NVARCHAR(20) DEFAULT 'Completed',
CreatedAt DATETIME DEFAULT GETDATE()
);
CREATE TABLE SaleItems (
Id INT PRIMARY KEY IDENTITY,
SaleId INT REFERENCES Sales(Id),
PartId INT REFERENCES Parts(Id),
Quantity INT NOT NULL,
UnitPrice DECIMAL(18,2) NOT NULL
);

```


GO

■ frontend/package.json

```
{
  "name": "autoparts-erp-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.7.2",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-router-dom": "^7.0.0",
    "recharts": "^2.12.7"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^6.0.0"
  }
}
```

■ frontend/vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:5151',
        changeOrigin: true
      }
    }
  }
})
```

■ frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>AutoParts ERP</title>
</head>
<body>
```

```

<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
</body>
</html>

```

■ frontend/src/main.jsx

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)

```

■ frontend/src/index.css

```

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
:root {
  --primary: #1e3a5f;
  --primary-light: #2563eb;
  --accent: #f59e0b;
  --success: #10b981;
  --danger: #ef4444;
  --bg: #f1f5f9;
  --sidebar-w: 240px;
}
body { font-family: 'Segoe UI', sans-serif; background: var(--bg); color: #1e293b; }
a { text-decoration: none; color: inherit; }
button { cursor: pointer; }

```

■ frontend/src/App.jsx

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { AuthProvider, useAuth } from './context/AuthContext'
import MainLayout from './layouts/MainLayout/MainLayout'
import Login from './pages/Login/Login'
import Dashboard from './pages/Dashboard/Dashboard'
import CustomerList from './pages/Customer/CustomerList'
import PartList from './pages/Parts/PartList'

function PrivateRoute({ children }) {
  const { user } = useAuth()
  return user ? children : <Navigate to="/login" />
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<Login />} />

```

```

<Route path="/" element={
  <PrivateRoute>
    <MainLayout />
  </PrivateRoute>
}>
<Route index element={<Dashboard />} />
<Route path="customers" element={<CustomerList />} />
<Route path="parts" element={<PartList />} />
{/* TODO: add more routes as modules are built */}
</Route>
</Routes>
</BrowserRouter>
</AuthProvider>
)
}

```

■ frontend/src/api/api.js

```

import axios from 'axios'

const api = axios.create({ baseURL: '/api' })

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
)

export default api

```

■ frontend/src/context/AuthContext.jsx

```

import { createContext, useContext, useState } from 'react'
import api from '../api/api'

const AuthContext = createContext(null)

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const token = localStorage.getItem('token')
    const name = localStorage.getItem('userName')
    return token ? { token, name } : null
  })

  const login = async (email, password) => {

```

```

const res = await api.post('/auth/login', { email, password })
const { token, fullName, role } = res.data
localStorage.setItem('token', token)
localStorage.setItem('userName', fullName)
setUser({ token, name: fullName, role })
}

const logout = () => {
  localStorage.clear()
  setUser(null)
}

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
)
}

export const useAuth = () => useContext(AuthContext)

```

■ frontend/src/layouts/MainLayout/MainLayout.jsx

```

import { Outlet } from 'react-router-dom'
import Sidebar from '../Sidebar/Sidebar'
import Navbar from '../Navbar/Navbar'
import './MainLayout.css'

export default function MainLayout() {
  return (
    <div className="layout">
      <Sidebar />
      <div className="layout-main">
        <Navbar />
        <main className="layout-content">
          <Outlet />
        </main>
      </div>
    </div>
  )
}

```

■ frontend/src/layouts/MainLayout/MainLayout.css

```

.layout { display: flex; min-height: 100vh; }
.layout-main { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
.layout-content { flex: 1; padding: 24px; overflow-y: auto; }

```

■ frontend/src/layouts/Sidebar/Sidebar.jsx

```

import { NavLink } from 'react-router-dom'
import './Sidebar.css'

const menu = [
  { label: 'Dashboard', path: '/', icon: '■' },

```

```

{ label: 'Customers', path: '/customers', icon: '■' },
{ label: 'Suppliers', path: '/suppliers', icon: '■' },
{ label: 'Parts', path: '/parts', icon: '■' },
{ label: 'Inventory', path: '/inventory', icon: '■' },
{ label: 'Purchase', path: '/purchase', icon: '■' },
{ label: 'Sales', path: '/sales', icon: '■' },
{ label: 'Expenses', path: '/expenses', icon: '■' },
{ label: 'Income', path: '/income', icon: '■' },
{ label: 'Ledger', path: '/ledger', icon: '■' },
{ label: 'Reports', path: '/reports', icon: '■' },
{ label: 'Settings', path: '/settings', icon: '■■' },
]

export default function Sidebar() {
  return (
    <aside className="sidebar">
      <div className="sidebar-logo">■■ AutoParts ERP</div>
      <nav>
        {menu.map(item => (
          <NavLink
            key={item.path}
            to={item.path}
            end={item.path === '/'}
            className={({ isActive }) => 'sidebar-link' + (isActive ? ' active' : '')}
          >
            <span>{item.icon}</span>
            <span>{item.label}</span>
          </NavLink>
        ))}
      </nav>
    </aside>
  )
}

```

■ frontend/src/layouts/Sidebar/Sidebar.css

```

.sidebar { width: var(--sidebar-w); background: var(--primary); color: #fff; display: flex;
flex-direction: column; }
.sidebar-logo { padding: 20px 16px; font-size: 1.1rem; font-weight: 700; border-bottom: 1px
solid rgba(255,255,255,0.1); }
.sidebar-link { display: flex; align-items: center; gap: 10px; padding: 12px 16px; color:
rgba(255,255,255,0.75); transition: background 0.2s; }
.sidebar-link:hover, .sidebar-link.active { background: rgba(255,255,255,0.12); color: #fff; }

```

■ frontend/src/layouts/Navbar/Navbar.jsx

```

import { useAuth } from '../../../context/AuthContext'
import './Navbar.css'

export default function Navbar() {
  const { user, logout } = useAuth()
  return (
    <header className="navbar">
      <span className="navbar-title">AutoParts ERP</span>
      <div className="navbar-right">
        <span>{user?.name}</span>
      </div>
    </header>
  )
}

```

```

<button onClick={logout} className="btn-logout">Logout</button>
</div>
</header>
)
}

```

■ frontend/src/layouts/Navbar/Navbar.css

```

.navbar { height: 56px; background: #fff; border-bottom: 1px solid #e2e8f0; display: flex;
align-items: center; justify-content: space-between; padding: 0 24px; }
.navbar-title { font-weight: 600; color: var(--primary); }
.navbar-right { display: flex; align-items: center; gap: 16px; }
.btn-logout { background: var(--danger); color: #fff; border: none; padding: 6px 14px;
border-radius: 6px; }

```

■ frontend/src/pages/Login/Login.jsx

```

import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '../../../context/AuthContext'
import './Login.css'

export default function Login() {
  const { login } = useAuth()
  const navigate = useNavigate()
  const [form, setForm] = useState({ email: '', password: '' })
  const [error, setError] = useState('')
  const [loading, setLoading] = useState(false)

  const handleSubmit = async (e) => {
    e.preventDefault()
    setError('')
    setLoading(true)
    try {
      await login(form.email, form.password)
      navigate('/')
    } catch {
      setError('Invalid email or password')
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="login-bg">
      <div className="login-card">
        <div className="login-logo">■■■</div>
        <h1>AutoParts ERP</h1>
        <p className="login-sub">Sign in to continue</p>
        {error && <div className="login-error">{error}</div>}
        <form onSubmit={handleSubmit}>
          <input
            type="email"
            placeholder="Email"
            value={form.email}
            onChange={e => setForm(f => ({ ...f, email: e.target.value })))}

```

```

required
/>
<input
type="password"
placeholder="Password"
value={form.password}
onChange={e => setForm(f => ({ ...f, password: e.target.value })))}
required
/>
<button type="submit" disabled={loading}>
{loading ? 'Signing in...' : 'Sign In'}
</button>
</form>
</div>
</div>
)
}

```

■ frontend/src/pages/Login/Login.css

```

.login-bg { min-height: 100vh; display: flex; align-items: center; justify-content: center;
background: var(--primary); }
.login-card { background: #fff; border-radius: 12px; padding: 40px; width: 360px; text-align:
center; box-shadow: 0 20px 60px rgba(0,0,0,0.3); }
.login-logo { font-size: 3rem; margin-bottom: 8px; }
h1 { color: var(--primary); margin-bottom: 4px; }
.login-sub { color: #64748b; margin-bottom: 24px; }
.login-error { background: #fef2f2; color: var(--danger); padding: 10px; border-radius: 6px;
margin-bottom: 16px; font-size: .875rem; }
input { display: block; width: 100%; padding: 10px 14px; margin-bottom: 12px; border: 1px solid
#e2e8f0; border-radius: 8px; font-size: 1rem; }
input:focus { outline: 2px solid var(--primary-light); border-color: transparent; }
button[type=submit] { width: 100%; padding: 11px; background: var(--primary); color: #fff;
border: none; border-radius: 8px; font-size: 1rem; font-weight: 600; }
button[type=submit]:hover { background: var(--primary-light); }

```

■ frontend/src/pages/Dashboard/Dashboard.jsx

```

import { useEffect, useState } from 'react'
import api from '../api/api'
import './Dashboard.css'

export default function Dashboard() {
const [data, setData] = useState(null)

useEffect(() => {
api.get('/dashboard').then(r => setData(r.data)).catch(() => {})
}, [])

if (!data) return <div className="loading">Loading dashboard...</div>

return (
<div>
<h2 className="page-title">Dashboard</h2>
<div className="stat-grid">
<StatCard label="Today's Sales" value={`■${data.today.sales.toLocaleString()}``>

```

```

color="success" icon="■" />
<StatCard label="Today's Purchase" value={`■${data.today.purchases.toLocaleString()}`}
color="primary" icon="■" />
<StatCard label="This Month Sales" value={`■${data.thisMonth.sales.toLocaleString()}`}
color="accent" icon="■" />
<StatCard label="Low Stock Parts" value={data.alerts.lowStock} color="danger" icon="■■" />
<StatCard label="Total Customers" value={data.totals.customers} color="primary" icon="■" />
<StatCard label="Total Suppliers" value={data.totals.suppliers} color="primary" icon="■" />
<StatCard label="Total Parts" value={data.totals.parts} color="success" icon="■" />
</div>
</div>
)
}

function StatCard({ label, value, color, icon }) {
return (
<div className={`stat-card stat-${color}`}>
<div className="stat-icon">{icon}</div>
<div>
<div className="stat-value">{value}</div>
<div className="stat-label">{label}</div>
</div>
</div>
)
}

```

■ frontend/src/pages/Dashboard/Dashboard.css

```

.page-title { font-size: 1.5rem; font-weight: 700; color: var(--primary); margin-bottom: 24px; }
.stat-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap: 16px; }
.stat-card { background: #fff; border-radius: 10px; padding: 20px; display: flex; align-items: center; gap: 16px; box-shadow: 0 1px 3px rgba(0,0,0,0.08); border-left: 4px solid transparent; }
.stat-success { border-color: var(--success); }
.stat-primary { border-color: var(--primary-light); }
.stat-accent { border-color: var(--accent); }
.stat-danger { border-color: var(--danger); }
.stat-icon { font-size: 2rem; }
.stat-value { font-size: 1.5rem; font-weight: 700; }
.stat-label { color: #64748b; font-size: .85rem; }
.loading { text-align: center; padding: 40px; color: #64748b; }

```

■ frontend/src/pages/Customer/CustomerList.jsx

```

import { useEffect, useState } from 'react'
import api from '../../api/api'

export default function CustomerList() {
const [customers, setCustomers] = useState([])
const [loading, setLoading] = useState(true)

useEffect(() => {
api.get('/customers')
.then(r => setCustomers(r.data))
.finally(() => setLoading(false))
}, [])
}

```

```

}, [])

if (loading) return <p>Loading...</p>

return (
  <div>
    <h2 className="page-title">Customers</h2>
    <table style={{ width: '100%', borderCollapse: 'collapse', background: '#fff', borderRadius: 8,
overflow: 'hidden' }}>
      <thead style={{ background: 'var(--primary)', color: '#fff' }}>
        <tr>
          {[ '#', 'Name', 'Phone', 'Email', 'Balance' ].map(h => (
            <th key={h} style={{ padding: '10px 14px', textAlign: 'left' }}>{h}</th>
          ))}
        </tr>
      </thead>
      <tbody>
        {customers.map((c, i) => (
          <tr key={c.id} style={{ borderBottom: '1px solid #e2e8f0' }}>
            <td style={{ padding: '10px 14px' }}>{i + 1}</td>
            <td style={{ padding: '10px 14px' }}>{c.name}</td>
            <td style={{ padding: '10px 14px' }}>{c.phone || '-'}</td>
            <td style={{ padding: '10px 14px' }}>{c.email || '-'}</td>
            <td style={{ padding: '10px 14px' }}>■{c.openingBalance}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
)
}

```

■ frontend/src/pages/Parts/PartList.jsx

```

import { useEffect, useState } from 'react'
import api from '../api/api'

export default function PartList() {
  const [parts, setParts] = useState([])
  const [search, setSearch] = useState('')
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    api.get('/parts')
      .then(r => setParts(r.data))
      .finally(() => setLoading(false))
  }, [])

  const filtered = parts.filter(p =>
    p.name.toLowerCase().includes(search.toLowerCase()) ||
    (p.partNumber || '').toLowerCase().includes(search.toLowerCase())
  )

  if (loading) return <p>Loading...</p>

  return (
    <div>

```

```

<h2 className="page-title">Parts</h2>
<input
  placeholder="Search by name or part number..."
  value={search}
  onChange={e => setSearch(e.target.value)}
  style={{ padding: '8px 14px', width: 300, borderRadius: 8, border: '1px solid #e2e8f0',
  marginBottom: 16 }}
/>
<table style={{ width: '100%', borderCollapse: 'collapse', background: '#fff', borderRadius: 8,
overflow: 'hidden' }}>
  <thead style={{ background: 'var(--primary)', color: '#fff' }}>
    <tr>
      {[ '#', 'Name', 'Part No.', 'Category', 'Brand', 'Cost', 'Sale', 'Stock', 'Status' ].map(h => (
        <th key={h} style={{ padding: '10px 14px', textAlign: 'left' }}>{h}</th>
      ))}
    </tr>
  </thead>
  <tbody>
    {filtered.map((p, i) => (
      <tr key={p.id} style={{ borderBottom: '1px solid #e2e8f0' }}>
        <td style={{ padding: '10px 14px' }}>{i + 1}</td>
        <td style={{ padding: '10px 14px' }}>{p.name}</td>
        <td style={{ padding: '10px 14px' }}>{p.partNumber || '-'}</td>
        <td style={{ padding: '10px 14px' }}>{p.category}</td>
        <td style={{ padding: '10px 14px' }}>{p.brand}</td>
        <td style={{ padding: '10px 14px' }}>■{p.costPrice}</td>
        <td style={{ padding: '10px 14px' }}>■{p.salePrice}</td>
        <td style={{ padding: '10px 14px' }}>{p.currentStock}</td>
        <td style={{ padding: '10px 14px' }}>
          <span style={{ color: p.lowStock ? 'var(--danger)' : 'var(--success)', fontWeight: 600 }}>
            {p.lowStock ? '■ Low' : '✓ OK'}
          </span>
        </td>
      </tr>
    ))}
  </tbody>
</table>
</div>
)
}

```